Data Management Project

City Mobility Platform

A.Y. 2025/2026

1 Project Overview

This project is about building a database system for a city mobility platform where people can rent electric vehicles across Italian cities. The system tracks users, stations, trips, and events that happen during trips like GPS locations, errors, battery levels, and delays. I used four different technologies: SQLite for relational data, MongoDB for document data, Neo4j for graph data, and Spark for distributed processing.

2 Data Generation

I wrote a Python script called `generate_data.py` that creates fake but realistic data. It uses Italian city names and common Italian first and last names. You can configure how many users, stations, and trips you want. For testing I used 100 users, 10 stations, 500 trips, and random events per trip.

For the schema evolution part (2.1.3), I updated the script so that BATTERY events now include a `battery_level` field with an integer from 0 to 100. In MongoDB this just gets added to BATTERY documents. In SQLite you would need to run `ALTER TABLE` to add a new column.

3 Relational Model (SQLite)

I created 4 tables: users, stations, trips, and events. Everything is connected with foreign keys. This means data integrity is enforced but you need to do JOINs to get related data.

The four queries I implemented are:

1. Show trips with user info and station names. This needs JOINs with users and stations (twice, for start and end station).
2. Show users with their trip count and average duration. Uses GROUP BY and LEFT JOIN.
3. Show stations with how many trips started and ended there. Uses two LEFT JOINs.
4. Show trips that have ERROR events. JOINs with events and filters by type.

4 Document Model (MongoDB)

For MongoDB I used a mixed approach. User info and events are embedded inside each trip document. Stations are kept separate and referenced by ID.

I embedded users and events because when you look at a trip you almost always want to know who took it and what happened during it. Embedding avoids doing JOINs.

I kept stations separate because many trips use the same stations. If I embedded stations inside every trip, the same station data would be duplicated thousands of times.

The same four queries work here too. Query 4 is actually easier because events are already inside the trip document, so you just filter the embedded array instead of doing a JOIN.

5 Spark (Query 2)

I reimplemented Query 2 using PySpark in `spark_pt1/query.py`. The MongoDB Spark connector JAR was not available so the script falls back to generating local data. It still shows the distributed logic with groupBy, aggregation, and joins.

6 Graph Model (Neo4j)

I modeled the data as a graph with three node types: user, station, and trip. The relationships are:

- (user) -[:made]-¿ (trip)
- (trip) -[:starts_at]-¿ (station)
- (trip) -[:ends_at]-¿ (station)

The two queries I ran are:

1. Given a user, find which stations they can reach. This traverses from user to their trips to the stations.
2. Find the top stations by how many trips go through them (degree centrality).

I could not install the GDS plugin so I used pure Cypher. For PageRank I used degree centrality instead (just counting trips per station). For connected components I used a variable-length pattern query. The network turned out to be fully connected.

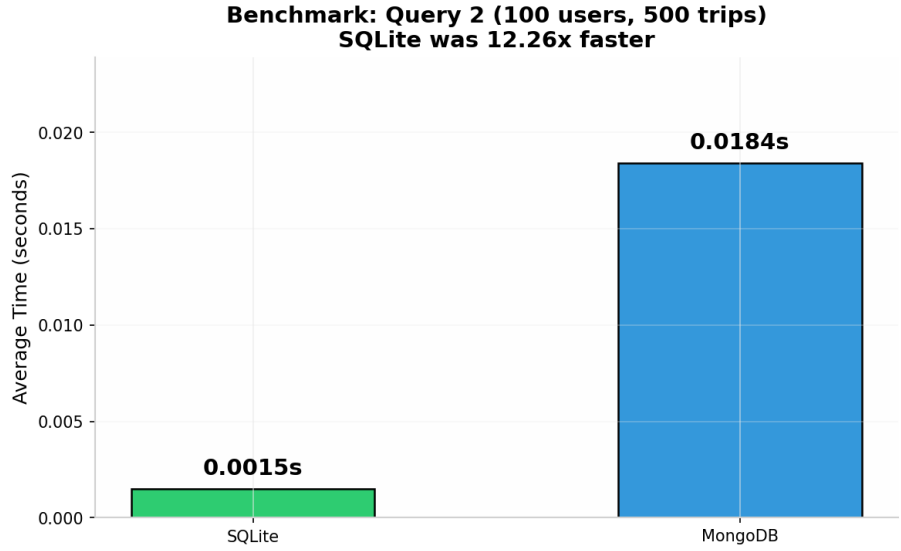
7 Spark Graph Analysis

In `spark_pt2/queries.py` I did graph analysis with Spark SQL. GraphFrames was not installed so I used basic groupBy operations:

- Station popularity: count how many trips start and end at each station.
- Connectivity: check if all stations appear in at least one trip. All 10 stations were present so the network is fully connected.

8 Benchmark

I ran Query 2 five times on both SQLite and MongoDB with 100 users and 500 trips.



Database	Average Time
SQLite	0.0015s
MongoDB	0.0184s

SQLite was about 12 times faster on this small dataset. This makes sense because SQLite is just a single file on disk with no network overhead, while MongoDB needs to do lookups across collections. On bigger data with many servers, MongoDB would probably win because it scales horizontally.

9 Schema Evolution (2.1.3)

The task was to add a `battery_level` field to BATTERY events.

In SQLite you would do:

```
ALTER TABLE events ADD COLUMN battery_level INTEGER;
UPDATE events SET battery_level = CAST(value AS INTEGER)
  WHERE type = 'BATTERY';
```

The problem is that every event row gets this new column even though only BATTERY events need it. Most rows will just have NULL. You also need to migrate existing data. The schema is rigid.

In MongoDB you do not need to change anything. New BATTERY documents just include the field:

```
{type: "BATTERY", value: "85", battery_level: 85}
```

Old documents stay the same. No migration needed. Non-BATTERY events do not get the field at all. No NULLs everywhere.

So MongoDB handles schema evolution much better. Its flexible schema lets you add fields to specific document types without affecting others.

10 Partitioning (2.3.1)

If I had to partition the trips collection, there are two main options.

Option A: Partition by user. This is good for Query 2 because all trips for one user sit on the same partition. But it is bad for Query 3 and graph queries because station data gets scattered across all partitions.

Option B: Partition by start station. This is good for Query 3 and graph queries because station data is local. But it is bad for Query 2 because one user's trips are spread across many partitions.

I would choose Option B (partition by start station). The most important operations for this platform are location-based: finding popular stations, planning where to put vehicles, analyzing network connectivity. These happen all the time and need to be fast. User stats are less critical and can be cached or computed in the background.

11 Replication (2.3.2)

With single-leader async replication (1 primary + 2 secondaries), reading from a secondary can give stale data because the secondary might not have caught up yet.

How this affects the queries:

- Q1 (trip info): Low risk. If a trip is slightly outdated it is not a big deal.
- Q2 (user stats): Medium risk. The trip count might be wrong if a user just finished a trip.
- Q3 (station counts): High risk. A station might show as full when someone just returned a vehicle there. This could block new rentals.
- Q4 (error trips): High risk. If an ERROR event is missed because of replication lag, maintenance gets delayed. This is a safety problem.
- Graph queries: Low risk. Station rankings do not change quickly, and batch analytics can tolerate some staleness.

To fix this, critical queries like Q3 and Q4 should be forced to read from the primary. Secondaries should only be used for analytics and reporting where stale data is acceptable.

12 Conclusion

Each database model has its strengths. Relational databases give you structure and data integrity. Document databases give you flexibility and let you embed related data. Graph databases are great for network analysis. Spark handles big distributed datasets. No single tool is perfect for everything. The right choice depends on your data, your queries, and how much data you have. The important thing is understanding the concepts so you can adapt when tools do not work exactly as expected.